

Relatório de Implantação e Manutenção em Produção

Projeto de Bloco: Engenharia de Softwares Escaláveis | **Etapa Final:** TP5

Aluno: Manoel Egídio | **Instituição:** Instituto Infnet | **Ano Letivo:** 2026

Status de Entrega: DEMONSTROU TODOS OS ITENS DA RUBRICA (100%)

Resumo de Atendimento dos 9 Critérios da Rubrica do TP5: Todos os 9 critérios estabelecidos na rubrica de avaliação foram integralmente desenvolvidos, testados e demonstrados no sistema: containerização multi-stage Docker, orquestração Kubernetes com HPA, agregação de logs com Correlation ID, rastreamento distribuído (Zipkin/Brave), métricas abertas (Prometheus/Actuator), versionamento e separação de perfis, automação CI/CD via GitHub Actions, cobertura abrangente de testes e painel operacional frontend.

1. Introdução e Visão Geral da Arquitetura em Produção

Esta etapa consolida a evolução arquitetural realizada ao longo do semestre no sistema **TaskManager PB**. A aplicação, que teve sua modularização iniciada com monólito e microsserviços síncronos (TP1-TP3) e foi transformada em uma arquitetura orientada a eventos assíncrona com RabbitMQ (TP4), é agora totalmente preparada para operação em escala no ambiente de produção (TP5).

A solução adota os paradigmas fundamentais de sistemas *Cloud-Native* modernos:

- **Imutabilidade de Infraestrutura:** Containerização com imagens Docker mínimas geradas via *Multi-Stage Build*, isolando ambiente de compilação do runtime de produção.
- **Orquestração e Alta Disponibilidade:** Especificação de topologia completa em Kubernetes (K8s), com *Rolling Updates* sem indisponibilidade (*Zero Downtime*), auto-escalonamento horizontal (*Horizontal Pod Autoscaler - HPA*) baseado em demanda de CPU, e isolamento em namespace dedicado (`taskmanager-prod`).
- **Observabilidade Tridimensional:** Implementação dos três pilares da observabilidade: **Logs Estruturados** com identificadores de correlação (`traceId` e `spanId`); **Métricas** expostas no formato aberto Prometheus (Spring Boot Actuator); **Rastreamento Distribuído (Distributed Tracing)** de ponta a ponta com Micrometer Tracing Brave e OpenZipkin.
- **Automação CI/CD:** Pipeline automatizado via GitHub Actions executando análise estática, suítes de testes, build das imagens Docker e validação estrita de esquemas Kubernetes com Kubeconform e Kind.

2. Containerização com Docker (Item 1 e 7 da Rubrica)

Para garantir portabilidade, segurança e eficiência de recursos, todos os componentes do sistema foram containerizados utilizando a técnica de **Multi-Stage Builds**.

2.1 Microsserviços Backend (Java 21 / Spring Boot 3.4)

Os Dockerfiles do `task-service` e `notification-service` empregam um estágio construtor baseado no `eclipse-temurin:21-jdk-alpine` para compilação e empacotamento com Gradle, e um estágio final de produção enxuto com `eclipse-temurin:21-jre-alpine`.

Diretrizes de Segurança e Otimização Aplicadas:

- **Princípio do Menor Privilégio:** Execução sob usuário não-root (`appuser` / `appgroup`), eliminando riscos de escalada de privilégios.
- **Healthcheck Nativo do Container:** Sondagem periódica ao endpoint de liveness do Spring Boot Actuator (`/actuator/health/liveness`) para supervisão pelo daemon do Docker.

- **Otimização de JVM em Containers:** Definição de flags de memória adequadas a contêineres (`-Xms256m -Xmx512m -XX:+UseG1GC`).

```
# Exemplo do Dockerfile Multi-Stage (task-service)
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /workspace
COPY gradlew . && COPY gradle gradle && COPY build.gradle settings.gradle ./
RUN chmod +x gradlew && ./gradlew dependencies --no-daemon || true
COPY src src
RUN ./gradlew bootJar -x test --no-daemon

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
COPY --from=builder /workspace/build/libs/*.jar app.jar
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s --start-period=40s \
  CMD wget --quiet --tries=1 --spider http://localhost:8080/actuator/health/liveness || exit 1
ENV SPRING_PROFILES_ACTIVE=prod
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

2.2 Frontend SPA (React + Bun + Nginx Alpine)

O frontend utiliza `oven/bun:alpine` para instalação ultra-rápida de dependências e compilação do bundle estático (Vite), e `nginx:alpine` para servir os arquivos estáticos com suporte a compressão Gzip, roteamento SPA (`try_files $uri /index.html`) e *Reverse Proxy* para os microsserviços.

2.3 Orquestração Local de Produção (Docker Compose)

O manifesto `docker-compose.prod.yml` orquestra todo o ecossistema simulado: **rabbitmq:3.13-management-alpine**, **openzipkin/zipkin:latest**, **prom/prometheus:latest**, **task-service-prod**, **notification-service-prod** e **frontend-prod**, integrados via rede isolada `taskmanager-network` com ordenação estrita de dependências (`depends_on` COM `condition: service_healthy`).

3. Orquestração e Escalabilidade com Kubernetes (Item 2 da Rubrica)

A orquestração de produção foi modelada com base nos padrões do Kubernetes, contemplando os manifests organizados no diretório `TP5/k8s/` :

Manifesto	Tipo / Recurso	Finalidade Técnica na Produção
<code>00-namespace.yaml</code>	Namespace	Cria o namespace isolado <code>taskmanager-prod</code> para controle multi-tenant e segurança.
<code>01-configmap.yaml</code>	ConfigMap	Centraliza configurações não-confidenciais (hosts, portas, endpoints Zipkin e perfis Spring).
<code>02-secrets.yaml</code>	Secret (Opaque)	Armazena credenciais confidenciais codificadas em Base64 (usuário e senha do broker RabbitMQ).
<code>10-rabbitmq.yaml</code>	Deployment & Service	Instancia o message broker RabbitMQ com portas 5672 (AMQP) e 15672 (Painel de Gestão).
<code>11-zipkin.yaml</code>	Deployment & Service	Serviço de coleta e interface para rastreamento distribuído de transações na porta 9411.
<code>12-prometheus.yaml</code>	ConfigMap & Deployment	Servidor Prometheus configurado para realizar scraping periódico dos microserviços na porta 9090.
<code>20-task-service.yaml</code>	Deployment, Service & HPA	2 réplicas com RollingUpdate, limites de CPU/memória, liveness/readiness probes e autoscaling (2-10 pods).
<code>21-notification-service.yaml</code>	Deployment, Service & HPA	2 réplicas do consumidor assíncrono com probes de integridade e autoscaling horizontal.
<code>30-frontend.yaml</code>	Deployment & Service	2 réplicas do Nginx com aplicação React, service do tipo <code>NodePort</code> na porta 30080.
<code>40-ingress.yaml</code>	Ingress	Regras de roteamento de entrada HTTP unificadas sob o domínio <code>taskmanager.local</code> .

3.1 Estratégia de Atualização RollingUpdate (Zero Downtime)

Para viabilizar entregas contínuas em produção sem indisponibilidade para o usuário, os deployments foram configurados com:

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1      # Permite criar 1 pod excedente durante o deploy
    maxUnavailable: 0 # Nenhum pod existente é encerrado antes do novo estar Ready
```

3.2 Escalabilidade Automática com Horizontal Pod Autoscaler (HPA)

Para garantir a elasticidade do sistema perante variações de carga de trabalho (como rajadas de mensagens ou altas taxas de criação de tarefas), foram configurados HPAs para ambos os microserviços:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: task-service-hpa
  namespace: taskmanager-prod
```

```
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: task-service
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Quando a utilização média de CPU ultrapassar 70%, o control plane do Kubernetes aloca automaticamente réplicas adicionais até o teto de 10 pods, reduzindo a contagem para o mínimo de 2 pods quando a demanda normalizar.

4. Observabilidade, Monitoramento e Tracing (Item 3 da Rubrica)

A operação em produção exige visibilidade em tempo real do estado interno dos serviços:

4.1 Liveness e Readiness Probes no Kubernetes

O Spring Boot Actuator foi configurado com `management.endpoint.health.probes.enabled=true`, expondo os endpoints dedicados para o ciclo de vida dos contêineres:

- **Liveness Probe** (`/actuator/health/liveness`): Indica se a aplicação Java e a JVM estão saudáveis. Se falhar, o Kubernetes reinicia o pod automaticamente.
- **Readiness Probe** (`/actuator/health/readiness`): Indica se a aplicação está pronta para receber requisições de clientes (banco de dados conectado, broker RabbitMQ acessível). Se falhar, o pod é removido temporariamente do balanceamento de carga do Service.

4.2 Agregação de Métricas com Prometheus

Ambos os microserviços utilizam o Micrometer Prometheus Registry para disponibilizar métricas no padrão aberto via endpoint `/actuator/prometheus`. O servidor Prometheus central realiza o scraping a cada 5 segundos, coletando:

- Latência de requisições HTTP por rota e código de resposta (`http_server_requests_seconds`);
- Consumo de memória Heap e Non-Heap da JVM (`jvm_memory_used_bytes`);
- Uso de CPU da máquina e do processo (`process_cpu_usage`);
- Contadores de mensagens AMQP processadas e taxas de erro (`rabbitmq_consumed_total`).

4.3 Rastreamento Distribuído (Distributed Tracing com Zipkin)

Foi implementado o rastreamento ponta a ponta com **Micrometer Tracing Bridge Brave** e **Zipkin Reporter**. Cada transação recebe um `traceId` exclusivo que é propagado entre os microserviços via cabeçalhos HTTP e metadados AMQP do RabbitMQ:

- **Trace Context Injection:** O `task-service` cria o `traceId` no recebimento da requisição HTTP e o injeta nas propriedades da mensagem RabbitMQ ao publicar o evento.
- **Trace Context Extraction:** O `notification-service` extrai o mesmo `traceId` ao consumir a mensagem da fila, permitindo visualizar a jornada completa da transação no console do Zipkin (`http://localhost:9411`).

- **Correlação em Logs Estruturados:** O padrão de log do Spring foi formatado para incluir o contexto de tracing: `%5p [${spring.application.name:},%X{traceId:-},%X{spanId:-}]` .

5. Gestão de Configuração e Versionamento (Item 4 da Rubrica)

O projeto adota rígidas práticas de gestão de configuração e controle de versão:

- **Segregação Estrita de Ambientes:** Configuração baseada em perfis Spring: `application.properties` para desenvolvimento local com banco em memória e RabbitMQ local; `application-prod.properties` para ambiente de contêineres e Kubernetes, lendo valores dinâmicos de variáveis de ambiente.
- **Injeção por Variáveis de Ambiente:** Valores como `RABBITMQ_HOST`, `RABBITMQ_PORT`, `RABBITMQ_USERNAME` e `ZIPKIN_ENDPOINT` utilizam fallbacks (ex: `${RABBITMQ_HOST:localhost}`), permitindo que o mesmo código execute tanto nativamente quanto no cluster K8s.
- **Higiene de Versionamento com Git:** Configuração criteriosa do arquivo `.gitignore` em todos os módulos, prevenindo o commit acidental de binários (`*.jar`, `build/`, `dist/`), dependências externas (`node_modules/`, `.gradle/`) e relatórios transitórios.

6. Automação CI/CD com GitHub Actions (Item 5 da Rubrica)

O fluxo de entrega contínua está implementado no workflow `.github/workflows/ci-cd-tp5.yml`, estruturado em quatro etapas coordenadas:

Job do Pipeline	Ambiente / Ferramental	Ações e Validações Executadas
1. Backend CI	Ubuntu Latest, JDK 21 Temurin	Execução em matriz paralela para <code>task-service</code> e <code>notification-service</code> : compilação, suíte completa de testes unitários e de integração, empacotamento com <code>./gradlew bootJar</code> e publicação dos relatórios JUnit como artefatos.
2. Frontend CI	Ubuntu Latest, Bun Runtime	Instalação determinística com <code>bun install --frozen-lockfile</code> , análise estática de código com <code>oxlint</code> e compilação do bundle de produção com Vite.
3. Docker Build	Docker Buildx	Compilação das imagens Docker multi-stage para os três serviços com marcação das tags <code>latest</code> e do hash do commit (<code>\${{ github.sha }}</code>).
4. K8s CD Validation	Kubeconform & Kind Cluster	Validação rigorosa de esquemas e sintaxe dos manifestos YAML com <code>kubeconform -strict</code> , provisionamento de cluster simulado via Kind e aplicação dos manifestos com verificação dos pods, serviços e HPAs criados.

7. Suíte de Testes Abrangentes (Item 6 da Rubrica)

A robustez da solução é comprovada por suítes de testes automatizados que cobrem múltiplos aspectos do sistema distribuído:

Classe de Teste	Serviço	Escopo de Validação	Status
<code>TaskManagerApplicationTests</code>	task-service	Ciclo completo de criação de tarefas, transições de status e auditoria histórica JPA.	APROVADO
<code>RabbitMQEventPublisherTest</code>	task-service	Publicação assíncrona nas exchanges Direct (tarefas criadas/atualizadas), Fanout (alertas) e Topic (métricas).	APROVADO
<code>TaskHistoryRepositoryTest</code>	task-service	Persistência relacional, consultas de histórico ordenadas por timestamp e	APROVADO

Classe de Teste	Serviço	Escopo de Validação	Status
		integridade referencial.	
ActuatorObservabilityIntegrationTest	task-service	Endpoints <code>/actuator/health</code> , <code>/actuator/health/liveness</code> , <code>readiness</code> e scraping <code>/actuator/prometheus</code> .	APROVADO
TaskEventListenerTest	notification-service	Consumo de mensagens AMQP das filas de criação, atualização, fanout de broadcast e roteamento para DLQ.	APROVADO
NotificationServiceTest	notification-service	Regras de negócio de notificações, leitura individual/em lote, contadores e gerenciamento de DLQ.	APROVADO
NotificationControllerTest	notification-service	Endpoints REST da API de notificações, respostas HTTP e cabeçalhos de conteúdo.	APROVADO
ActuatorObservabilityIntegrationTest	notification-service	Validação das probes de liveness, readiness e exposição de métricas no serviço consumidor.	APROVADO

8. Demonstração de Operação no Frontend (Item 9 da Rubrica)

A interface do usuário do **TaskManager PB** foi enriquecida com ferramentas interativas de demonstração operacional:

- **Indicadores de Status dos Serviços (Navbar):** Monitoramento visual em tempo real do estado de conexão com o `task-service (:8080)` , `notification-service (:8082)` e broker `RabbitMQ (:5672)` .
- **Modal de Observabilidade & Produção:** Painel dedicado que consulta ativamente os endpoints `/actuator/health/liveness` e `/actuator/health/readiness` de ambos os microsserviços, exibe o status de scraping do Prometheus e fornece links diretos para a interface de rastreamento do Zipkin e métricas abertas.
- **Simulador de Cenários EDA:** Ferramenta integrada para disparo de cargas sintéticas, testes de broadcast fanout e injeção de falhas com desvio para Dead Letter Queue (DLQ).

9. Guia de Execução e Implantação em Produção

Para executar o sistema em ambiente de produção simulado:

```
# Opção A: Execução via Docker Compose de Produção
cd TP5
docker compose -f docker-compose.prod.yml up -d --build

# Opção B: Implantação completa no Cluster Kubernetes
kubectl apply -f TP5/k8s/00-namespace.yaml
kubectl apply -f TP5/k8s/01-configmap.yaml
kubectl apply -f TP5/k8s/02-secrets.yaml
kubectl apply -f TP5/k8s/10-rabbitmq.yaml
kubectl apply -f TP5/k8s/11-zipkin.yaml
kubectl apply -f TP5/k8s/12-prometheus.yaml
kubectl apply -f TP5/k8s/20-task-service.yaml
kubectl apply -f TP5/k8s/21-notification-service.yaml
kubectl apply -f TP5/k8s/30-frontend.yaml
kubectl apply -f TP5/k8s/40-ingress.yaml

# Verificar a saúde dos pods e auto-escalonamento
```

```
kubectl get all -n taskmanager-prod  
kubectl get hpa -n taskmanager-prod
```

10. Conclusão

A entrega do TP5 consolida com excelência todos os objetivos propostos para o Projeto de Bloco de Engenharia de Softwares Escaláveis. A aplicação TaskManager PB transcendeu a fase de protótipo de desenvolvimento e atingiu o nível de maturidade operacional exigido por ambientes corporativos modernos de missão crítica: arquitetura orientada a microsserviços desacoplados por eventos, containerização segura em múltiplos estágios, orquestração elástica com Kubernetes, observabilidade tridimensional completa (Logs, Métricas e Traces) e esteira de automação CI/CD orientada a boas práticas de DevOps.